

プログラミング入門2

第10回 動的な領域確保、 共用体、列挙体

篠埜 功

今日の内容

- callocによる動的な領域確保について
 - mallocという、callocに似たライブラリ関数もあるが、この演習ではcallocのみ紹介する。
 - 動的な領域確保の典型例: リスト
- 共用体
- 列挙体

動的な記憶域確保

- これまでの方法 静的(static)な記憶域確保
 - 配列の要素数は固定。
 - あらかじめ十分な大きさの配列を確保しておく必要があった。
- 今回紹介する方法 動的(dynamic)な記憶域確保
 - 問題に応じて、適切なサイズの配列を確保するには、プログラムの実行時に確保を行う必要がある。
 - 余分なメモリの使用を避けることができる。

静的(static) --- プログラムのコンパイル時
動的(dynamic) --- プログラムの実行時

ヒープ領域(heap)

- プログラムからはヒープ領域(heap)を用いることができる。
- ヒープ領域を使うには、mallocあるいはcallocというライブラリ関数を呼び出すことにより領域を確保する。使い終わったら、freeというライブラリ関数を呼び出すことにより解放する。解放することにより、それ以降のmallocあるいはcallocの呼び出し時に再利用可能になる。

(注意)ヒープ領域は、データ構造の授業で習う木構造のヒープとは関係がない。

calloc関数

- ヒープ領域から実行時に記憶域を確保する。
- 引数として、データ型のサイズsize(第2引数)と、その個数n(第1引数)を受け取り、1つの要素の大きさがsizeで長さnの配列の領域を確保する。確保した領域のすべてのビットが0で初期化される。配列の確保に成功した場合は、その配列の先頭要素へのポインタを返し、失敗した場合は、ヌルポインタを返す。返り値の型はvoid * 型である。返り値をポインタ型の変数に代入するとき、キャストする必要はない(キャストしてもよいが)。
- データ型のサイズは、sizeof (型式) で取得できる。
- calloc関数を使うためにはstdlib.hをインクルードする必要がある。

ヌルポインタ(空ポインタ)

ヌルポインタ(null pointer)は、どこも指さないポインタであり、何かを指しているポインタとは異なることが保証されている。整数値0は、任意のポインタ型へキャストすることができ、その結果をヌルポインタという。

ヌルポインタを表すため、ヌルポインタ定数(0か、あるいは `(void *) 0`)がマクロNULLとして `stddef.h` に定義されている。(`stdio.h`, `stdlib.h`, `string.h`, `time.h` のいずれをincludeしても NULL が使える。)

ヌルポインタは、キャスト無しで任意のポインタ型の変数に代入したり、任意のポインタ型の値と比較してよい。自動的に型変換される(暗黙の型変換)。

例(打ち込んで確認)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(void)
```

```
{
```

```
    int *p;
```

```
    p = calloc (1, sizeof (int));
```

```
    if( p == NULL )
```

```
        printf ("記憶域の確保に失敗しました。¥n");
```

```
    else {
```

```
        *p = 15;
```

```
        printf ("*p = %d¥n", *p );
```

```
    }
```

```
    return 0;
```

```
}
```

int型1個分の記憶域(長さ1の
int型の配列)をヒープ領域から
割り当てる

callocの戻り値はキャスト
無しでpに代入してよい

NULLはキャスト無しでpと比較してよい

解説

- calloc関数による記憶域の動的な確保

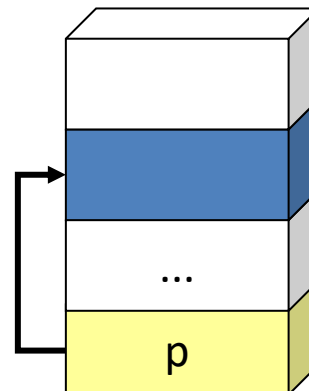
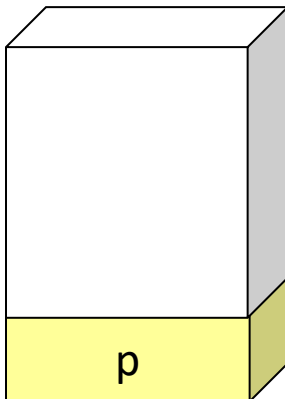
```
int * p;  
p = calloc (1, sizeof (int) );
```

int型へのポインタ型の
変数を宣言

```
int * p;
```

calloc関数呼び出し時にint型の長さ1の配列の領域が確保され、その先頭要素へのポインタがpに代入される。

```
p = calloc (1, sizeof (int) );
```



sizeof演算子

sizeof演算子は、型式(type expression)を引数にとる。評価結果は、その型のサイズである。

構文

sizeof (型式)

意味

sizeof (t) の評価結果はtのサイズである

型式は、int, double, char等の基本型、int [3]等の配列型、struct {int px; int py;}等の構造体型、int *等のポインタ型、typedefで定義した型名、あるいはこれらの組み合わせなどである。

例(打ち込んで確認)

```
#include <stdio.h>
typedef struct {
    int x;
    int y;
} point;
int main (void) {
    printf ("int: %d¥n", sizeof(int));
    printf ("int[3] : %d¥n", sizeof(int[3]));
    printf ("struct {int x; int y;} : %d¥n",
            sizeof(struct {int x; int y;}));
    printf ("point: %d¥n", sizeof(point));
    printf ("point *: %d¥n", sizeof(point *));
    return 0;
}
```

void へのポインタ型

calloc関数の返り値はvoid *型（voidへのポインタ型）である。

void *型のポインタを他のポインタ型変数に代入したり、他のポインタ型のポインタをvoid *型の変数に代入したりできる（暗黙の型変換が行われるのでキャストは不要）。

int, char, double, 構造体 など、さまざまな型の配列の領域を確保するためにcalloc関数が用いられるので、void *型で返している。

（キャストしない例）int *p;

p = calloc (1, sizeof (int));

（キャストする例）int *p;

p = (int *) calloc (1, sizeof (int));

（補足）C++では、void*型のポインタを他のポインタ型の変数に代入するときにはキャストが必要。

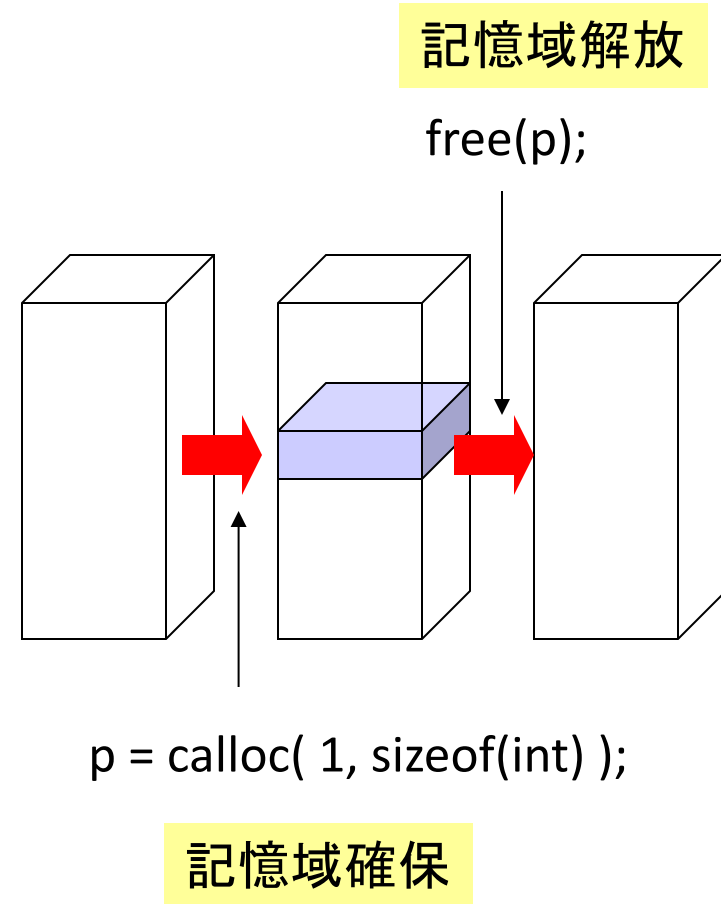
free関数：記憶域の解放

- 動的に確保した記憶域は、不要になった時点でfree関数を呼び出して解放する。それによって、それ以降のcallocあるいはmallocの呼び出しで再利用可能な状態になる。
- stdlib.hというヘッダーファイルを読み込んで使う。
- 引数にポインタpを受け取り、pが指す先の領域を解放する。返り値はない。ただし、pがヌルポインタのときは何も行わない。pはcalloc, malloc, あるいはreallocによって以前に割り当てられた領域へのポインタでなければならない(もしそうでない場合は動作は未定義)。pが、freeやreallocによって既に解放された領域を指している場合も動作は未定義。

(注意)reallocは、解放と割り当ての両方を行うライブラリ関数である。この演習では、malloc, reallocの説明はしない。

例

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int *p;
    p = calloc( 1, sizeof(int) );
    if (p == NULL)
        printf ("記憶域の確保に失敗しました。¥n");
    else {
        *p = 15;
        printf("*p = %d¥n", *p );
        free(p);
    }
    return 0;
}
```



確保した領域へキーボードからの入力を書き込む例 (打ち込んで確認)

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int * p;
    p = calloc (1, sizeof(int));
    if(p == NULL)
        printf ("記憶域の確保に失敗しました。¥n");
    else {
        printf ("整数を入力して下さい:");
        scanf ("%d", p);
        printf ("*p = %d¥n", *p);
        free(p);
    }
    return 0;
}
```

1次元配列の動的確保

- 微妙な配列宣言の例

```
void f (int n) {  
    int a[n];  
    ...  
}
```

配列の要素数を変数とすること(可変長配列)はC89では許されていない。
(注1)C99では可変長配列が使える。
(注2)C11では可変長配列はオプションになった。(つまり可変長配列は使えない場合もある)
(教科書p. 136参照)

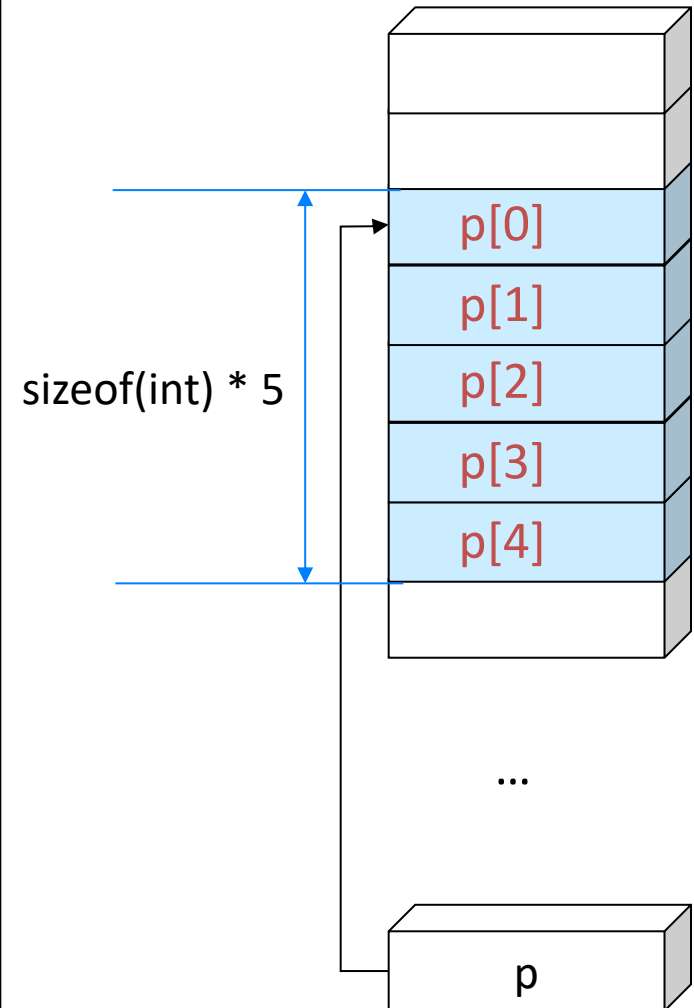


このようなことをしたければ、実行時にヒープ領域から配列を確保する。

例(打ち込んで確認)

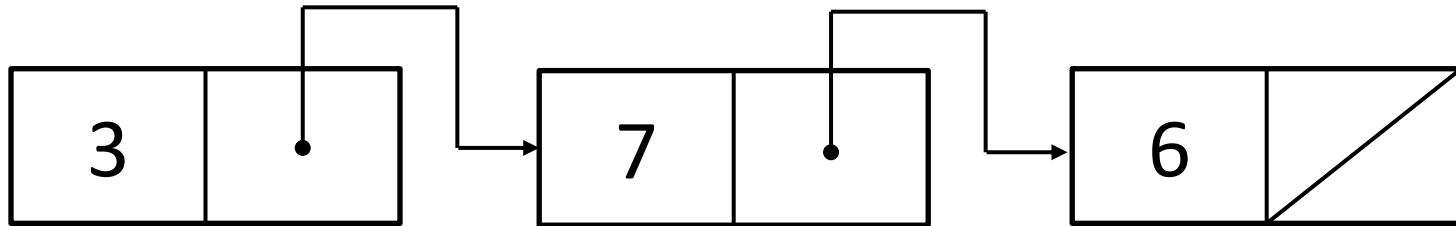
```
#include <stdio.h>
#include <stdlib.h>
int main (void) {
    int no, i=0;
    int * p;
    printf("確保する配列の要素数:");
    scanf("%d", &no);
    p = calloc (no, sizeof (int));
    if (p == NULL)
        printf ("記憶域の確保に失敗しました。¥n");
    else {
        for (i=0; i < no; i=i+1)
            p[i] = i;
        for (i=0; i < no; i=i+1)
            printf("p[%d] = %d¥n", i, p[i] );
        free (p);
    }
    return 0;
}
```

(5を入力した場合)



リスト(新・明解C言語実践編12章参照)

- リストとは、データが一行に連なっているデータ構造である。

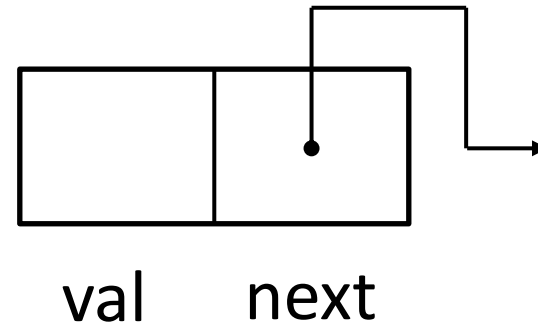


リストは、配列と異なり、要素がメモリ中の連続した領域に保存されるわけではなく、ある要素の次の要素はポインタで指す。リストは、一行のデータを作成した後で要素の追加や削除をする場合に用いる。

詳しくは2年生のデータ構造とアルゴリズムで学習してください。

リストの要素1つ分のデータ構造

```
struct node {  
    int val;  
    struct node * next;  
}
```



構造体の回で紹介しませんでしたでしたが、構造体型の宣言に2通りあり、上記は名前付きでの構造体型の書き方です(教科書p.332参照)。この例では、「struct node」がこの構造体型を表しています。以下のようにしてtypedefでこの構造体型に別の名前をつけてもいいです。

```
typedef struct node_tmp {  
    int val;  
    struct node_tmp * next;  
} node;
```

リストの例

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node_tmp {
    int val;
    struct node_tmp * next;
}node;

node * cons (int x, node * list) {
    node * p = calloc(1, sizeof(node));
    p->val = x;
    p->next = list;
    return p;
}
```

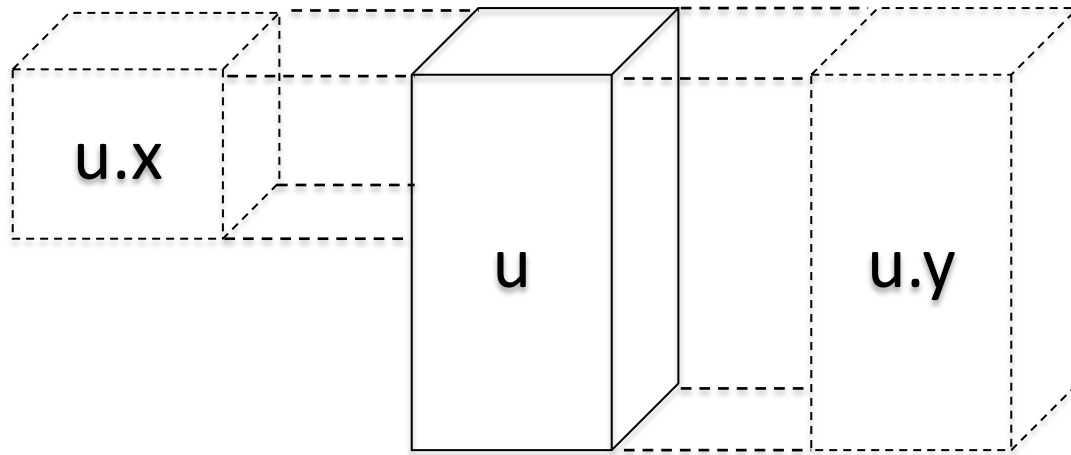
```
void printList (node * p) {
    while (p!=NULL) {
        printf ("%d ", p->val);
        p=p->next;
    }
    printf ("%n");
    return;
}

int main (void) {
    node * list1 = cons(3, cons(7, cons(6, NULL)));
    node * list2 = cons(5, cons(7, cons(8, NULL)));
    printList (list1);
    printList (list2);
    return 0;
}
```

新・明解C言語実践編12章のサンプルコードList12-1では二重のポインタを使っているが、ここではより単純な関数型でのプログラミング例を示している(命令型も混ざっている)。

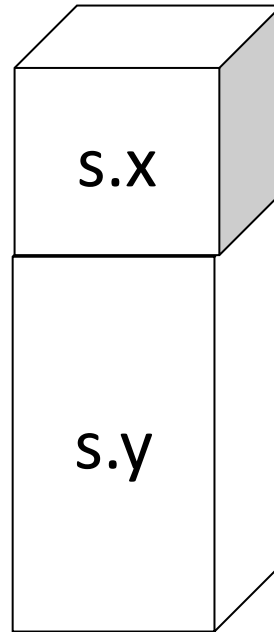
共用体

```
union {  
    int x;  
    double y;  
} u;
```



構造体（復習）

```
struct {  
    int x;  
    double y;  
} s;
```



共用体型を表す型式

共用体型を表す式は以下のような形で記述する。

(例)

```
union {  
    int x;  
    double y;  
}
```

共用体型を表す型式の構文

```
union {  
    変数宣言  
    変数宣言  
    ...  
}
```

キーワードunion の後、中括弧の中に
変数宣言を複数個並べる。

共用体型の変数の宣言

共用体型の変数を宣言できる。int型、double型、構造体型の変数宣言と同様、型式の後に変数名を書き、セミコロンを書いて宣言する。変数が複数の場合はコンマで区切って並べればよい。

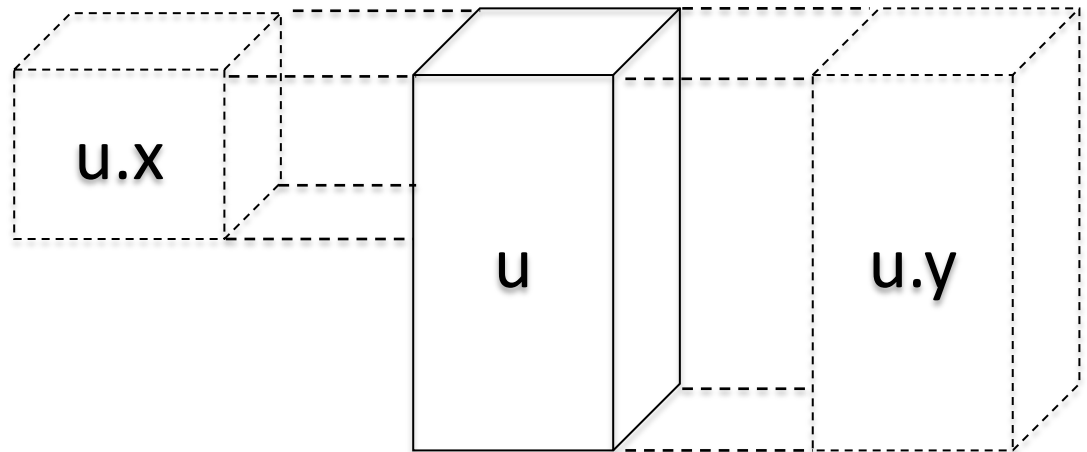
τ 変数名, ..., 変数名; (τ は共用体型を表す型式)

赤字の部分は共用体型を表す型式

(例)

```
union {  
  int x;  
  double y;  
} u;
```

↑
青字の部分は宣言
する変数名



共用体のメンバー

前ページの例:

```
union {  
    int x;  
    double y;  
} u;
```

x, yを、共用体uのメンバーという。

共用体のメンバーアクセス

式 e が、名前 m のメンバーを持つ共用体型の式するとき、 $e.m$ で共用体のメンバーが得られる。 $.$ をドット演算子と呼ぶ。

(例) 前ページのように u を宣言すると、 $u.x$, $u.y$ で共用体 u の各メンバーが得られる。

(注意) 共用体にどのメンバーの値が入っているかを認識してプログラムを書く必要がある。前のページの共用体 u で、メンバー y が入っている状態においてメンバー x にアクセスすることもできる。

例(打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    union {
        int x;
        double y;
    } u;
    u.x = 100;
    printf ("u.x=%d, u.y=%f¥n", u.x, u.y);
    u.y= 2.1;
    printf ("u.x=%d, u.y=%f¥n", u.x, u.y);
    return 0;
}
```

共用体の初期化

共用体の初期化は、構造体と同様、中括弧を用いるが、初期化は先頭のメンバに対して行われる。

```
#include <stdio.h>
int main (void) {
    union {
        int x;
        double y;
    } u = {100};
    printf ("u.x=%d, u.y=%f¥n", u.x, u.y);
    return 0;
}
```

共用体型に名前をつける例

これまでと同様、typedefにより共用体型に名前をつけることができる。

```
#include <stdio.h>
int main (void) {
    typedef union {
        int x;
        double y;
    } uxy;
    uxy u = {100};
    printf ("u.x=%d, u.y=%f¥n", u.x, u.y);
    return 0;
}
```

共用体の代入

同じ型の共用体であれば、代入することが可能

```
#include <stdio.h>
int main (void) {
    typedef union {
        int x;
        double y;
    } uxy;
    uxy u1, u2;
    u1.x = 100;
    u2 = u1;
    printf ("u1.x=%d, u1.y=%f¥n", u1.x, u1.y);
    printf ("u2.x=%d, u2.y=%f¥n", u2.x, u2.y);
    return 0;
}
```

その他

- 構造体と同様、共用体も関数へ渡したり、関数の返り値としたりできる。共用体へのポインタも使うことができ、それを関数へ渡すこともできる。
- アロー演算子 `->` が構造体と同じように使える。

列挙体

犬、猫、猿など、いくつかのものを名前付きで扱いたい場合に用いる。

(例) `enum {Dog, Cat, Monkey}`

`enum`の後に中括弧で名前をコンマで区切って並べる。`Dog, Cat, Monkey`のような名前を列挙定数という。それぞれの列挙定数に対し、書かれている順に0から順番に`int`型の値が割り当てられる。

例（打ち込んで確認）

```
#include <stdio.h>
int main (void) {
    enum {Dog, Cat, Monkey};
    printf ("%d, %d, %d¥n", Dog, Cat, Monkey);
    return 0;
}
```

Dogが0, Catが1, Monkeyが2になる。

数を指定する例

```
#include <stdio.h>
int main (void) {
    enum {Dog, Cat=2, Monkey};
    printf ("%d, %d, %d¥n", Dog, Cat, Monkey);
    return 0;
}
```

enumの後の中括弧の中で、列挙定数に割り当てる数を上記のように指定することができる。負の数を指定してもよい。指定がない場合は、2番目以降の場合は左隣の列挙定数に割り当てられている数に1を足した数が割り当てられ、1番目の場合は0が割り当てられる。複数箇所で数を指定してよく、同じ数が複数の列挙定数に割り当てられることになってもよい。

列挙体型の変数の宣言

```
#include <stdio.h>
int main (void) {
    enum {Dog, Cat, Monkey} a,b,c;
    a=Dog;
    b=Cat;
    c=Monkey;
    printf ("a=%d, b=%d, c=%d¥n", a,b,c);
    return 0;
}
```

int型、double型や構造体型、共用体型と同様、型式の後に変数名をコンマで並べてセミコロンをつければよい。a,b,cは上記のように宣言する代わりにint型として宣言してもよいが、列挙体型で宣言してあれば処理系によっては範囲外の値の代入に対して警告を発してくれる場合がある。

typedefを使う場合

```
#include <stdio.h>
int main (void) {
    typedef enum {Dog, Cat, Monkey} animal;
    animal a,b,c;
    a=Dog;
    b=Cat;
    c=Monkey;
    printf ("a=%d, b=%d, c=%d\n", a,b,c);
    return 0;
}
```

基本課題1

文字列(アルファベットのみ)をキーボードから受け取り、それを逆順に表示するプログラムを作成せよ。文字列を格納する領域は、キーボードから文字数の上限を受け取り、callocで確保せよ。(上限以上の文字が入力された場合の対処は自由とする。)

(注意)文字列の形で格納する場合、最後にヌル文字が必要である。ただ、この問題では逆順に表示できさえすればよく、ヌル文字を追加で格納するかどうかは自由とする。

[実行例]

文字数の上限を入力してください: 10

文字列を入力してください: abcde

edcba

基本課題2

以下のように学部生か大学院生かをキーボードから読み取り、それを画面に表示するプログラムを作成せよ。ただし、学部生か大学院生かは以下のように定義される列挙体型ugを用いて表し、ug型の数値を受け取って画面に表示する関数showUGを定義してそれを用いたプログラムとせよ。

```
typedef enum {Und, Gra} ug;  
void showUG (ug s) { ... }
```

[実行例]

学部生か大学院生かを入力(学部生0, 大学院生1): 0
あなたは学部生です。

(注意) 上記のtypedef宣言はプログラムの先頭部分(showUG関数, main関数より上の部分)で宣言する必要がある。

発展課題1

受験者数、およびその人数分の氏名、数学、英語の点数を標準入力から受け取り、氏名、各科目の点数の一覧を表示するプログラムをsample.cとして講義用ページに置いてある。このプログラムを、以下の実行例のように合計点も表示されるように修正せよ。データの例はseiseki.txtとして講義用ページに置いてある。

(実行例) % ./a.out < seiseki.txt

氏名	数学	英語	合計
芝浦太郎	90	90	180
芝浦次郎	100	100	200
芝浦三郎	80	90	170
芝浦四郎	77	88	165
芝浦五郎	50	60	110
芝浦六郎	30	35	65
芝浦七郎	40	40	80
芝浦八郎	55	56	111
芝浦九郎	77	78	155
芝浦十郎	88	99	187

一覧表示で縦をそろえるには、printfの変換指定を、文字列の場合は%-8s, 整数の場合は%5dのようにすればよい。
詳しくは教科書p.354-357を参照。
あるいはmanコマンドで
\$ man -S 3 printf
で調べればよい。

発展課題2

発展課題1の表示を、合計点の高い順に表示するように変更せよ。

発展課題3

英文をキーボードから入力し、その英文中の単語の数を表示するプログラムを作成せよ。英字以外の文字(空白、ピリオド、コンマ、クエスチョンマーク、ハイフン、アポストロフィー等)は区切り文字とし、単語数にはカウントしない。文字列を格納する領域は、キーボードから文字数の上限を受け取り、callocで確保せよ。

[実行例]

% ./a.out

文字数の上限を入力してください: 20

英文を入力してください: This is a pen.

単語数は4です。

(参考)これは第8回発展課題1の類題で、文字列を格納する領域をcallocで確保するようにした問題

発展課題4

以下のように生まれた月をキーボードから読み取り、それを英語で画面に表示するプログラムを作成せよ。ただし、月は以下のように定義される列挙体型monthを用い、month型の数値を受け取り、月を英語で画面に表示する関数showMonthを定義してそれを用いたプログラムとせよ。(showMonthで英文全体を表示してください。)

```
typedef enum{Jan=1, Feb, Mar, Apr, May, Jun, Jul, Aug,  
Sep, Oct, Nov, Dec} month;  
void showMonth (month m) { ... }
```

[実行例]

生まれた月を入力: 7

You were born in July.

(注意) 上記のtypedef宣言はプログラムの先頭部分(showMonth関数, main関数より上の部分)で宣言する必要がある。

発展課題5

円の半径か三角形の底辺と高さをキーボードから読み取り、面積を小数点以下1桁まで表示するプログラムを作成せよ。ただし、円か三角形は以下の型ct、図形情報は以下の型figを用いて表し、fig *型を受け取って面積をdouble型で返す関数areaを定義してそれを用いたプログラムとせよ。円周率は3.14とする。

```
typedef enum {Circle, Triangle} ct;
typedef struct {
    ct ct;
    union {
        double radius;
        struct {
            double teihen;
            double takasa;
        } tri;
    } info;
} fig;
double area (fig * fig) { ... }
```

[実行例]

```
% ./a.out
```

円か三角形を選択(円0, 三角形1): 0

半径: 3.5

面積は38.5です。

```
% ./a.out
```

円か三角形を選択(円0, 三角形1): 1

底辺: 2.5

高さ: 3.0

面積は3.8です。

(注意) 上記のtypedef宣言はプログラムの先頭部分(area関数, main関数より上の部分)で宣言する必要がある。

発展課題6(2年生の内容なのでできる人 だけでokです)

リストの例のプログラムに、2つのリストを受け取ってそれらを連結したリストを返す関数appendを以下の形で追加せよ。

```
node * append (node * p1, node * p2) {  
    ...  
}
```

第2引数に与えられたリストはそのまま再利用することを条件とする。さらに、main関数内で `printList(append(list1, list2));` を呼び出すことにより動作を確認せよ。

[実行例]

```
% ./a.out  
3 7 6  
5 7 8  
3 7 6 5 7 8  
%
```

scanfについて

```
int n;  
char c;  
scanf ("%d", &n);  
scanf ("%c", &c);
```

%cは空白や改行文字も読み取り対象となるので、%dで読み取ったあとに%cで読み取る場合は注意が必要。

のようなプログラムにおいて、例えば5を入力すると、nに5が代入されるが、入力のためにreturnキーを押しており、改行文字が残っているため、scanf("%c", &c);で改行文字が読み取られる。

次の文字を読み取るためには、以下のようにscanf ("%c", &c); を間に入れて改行文字を読み捨てればよい。

```
scanf ("%d", &n);  
scanf ("%c", &c);  
scanf ("%c", &c);
```

scanfについて(続き)

```
int n;  
char c[20];  
scanf ("%d", &n);  
fgets (c, 20, stdin);
```

のようなプログラムにおいて、例えば5を入力すると、nに5が代入されるが、入力のためにreturnキーを押しており、改行文字が残っているため、fgets (c, 20, stdin);で改行文字だけが読み取られる。

入力文字列を読み取るためには、以下のようにscanf ("%c", c);を間に入れて改行文字を読み捨てればよい。

```
scanf ("%d", &n);  
scanf ("%c", c);  
fgets (c, 20, stdin);
```

scanfについて(続き)

scanfで%dが指定されている場合は、数が出てくるまで、改行や空白が読み飛ばされる。

```
int n1;
```

```
int n2;
```

```
scanf ("%d", &n1);
```

```
scanf ("%d", &n2);
```

のようなプログラムだと、1回目のscanfで数を入れた後は改行文字が残っているが、次のscanfでは残っていた改行文字は読み飛ばされ、その後の数字が読み取られる。(その数字の後に改行文字があったらそれは残る。)

参考課題1

n個 (nは実行時にキーボードから入力) のint型の数をキーボードから受け取り、それらの和を画面上に表示するプログラムを作成せよ。ただし、callocを用いて長さnのint型の領域を確保し、そこへキーボードからのn個の入力を格納せよ。和を計算する部分は、その領域の先頭要素へのポインタおよび長さnを受け取って和を返す以下のような関数として定義せよ。

```
int sum (int * p, int n) { ... }
```

[実行例]

いくつ入力しますか: 5
1個目の数字を入力: 3
2個目の数字を入力: 6
3個目の数字を入力: 1
4個目の数字を入力: 8
5個目の数字を入力: 7
合計は25です

参考課題1

解答例

```
#include<stdio.h>
#include<stdlib.h>

int sum (int * p, int n) {
    int i, sum=0;
    for (i=0; i<n; i++)
        sum = sum + p[i];
    return sum;
}
```

```
/* 続き */
int main (void) {
    int n,i;
    int * p;
    printf("いくつ入力しますか: ");
    scanf("%d", &n);
    p = calloc (n, sizeof (int));
    if (p == NULL)
        printf ("記憶域の確保に失敗しました。¥n");
    else {
        for(i=0; i<n; i++){
            printf("%d個目の数字を入力: ", i+1);
            scanf("%d", &p[i]);
        }
        printf("合計は%dです¥n",sum(p,n));
        free (p);
    }
    return 0;
}
```

参考課題2

季節を以下の実行例のようにキーボードから入力し、それを表示するプログラムを作成せよ。ただし、季節は以下のseason型で表すようにせよ。

```
typedef enum {Spring, Summer, Autumn, Winter} season;
```

[実行例]

```
% ./a.out
```

```
季節を入力(春0, 夏1, 秋2, 冬3): 2  
秋です。
```

参考課題2の解答例

```
#include <stdio.h>
int main (void) {
    enum {Spring, Summer, Autumn, Winter};
    int s;
    printf ("季節を入力(春0, 夏1, 秋2, 冬3): ");
    scanf ("%d", &s);
    if (s==Spring)
        printf ("春");
    else if (s==Summer)
        printf ("夏");
    else if (s==Autumn)
        printf ("秋");
    else if (s==Winter)
        printf ("冬");
    printf ("です。¥n");
    return 0;
}
```

参考課題3

円の半径か長方形の縦と横の長さをキーボードから読み取り、それらの情報を表示するプログラムを作成せよ。ただし、円か長方形かは以下の型cr、半径等の情報は以下の型infoを用いて表すようにせよ。

```
typedef enum {Circle, Rectangle} cr;
typedef union {
    double radius;
    struct {
        double tate;
        double yoko;
    } rect;
} info;
```

[実行例]

```
% ./a.out
```

```
円か長方形を選択(円0, 長方形1): 0
```

```
半径: 3.5
```

```
円の半径は3.500000です。
```

```
% ./a.out
```

```
円か長方形を選択(円0, 長方形1): 1
```

```
縦: 2.5
```

```
横: 3.0
```

```
長方形の縦は2.500000, 横は3.000000です。
```

参考課題3の解答例

```
#include <stdio.h>
enum {Circle, Rectangle};
typedef union {
    double radius;
    struct {
        double tate;
        double yoko;
    } rect;
} info;
```

```
/* 続き */
int main (void) {
    info info;
    int cr;
    printf ("円か長方形を選択(円0, 長方形1): ");
    scanf ("%d", &cr);
    if (cr == Circle) {
        printf ("半径: ");
        scanf ("%lf", &info.radius);
        printf ("円の半径は%fです。¥n", info.radius);
    }
    else if (cr == Rectangle) {
        printf ("縦: ");
        scanf ("%lf", &info.rect.tate);
        printf ("横: ");
        scanf ("%lf", &info.rect.yoko);
        printf ("長方形の縦は%f, 横は%fです。¥n",
            info.rect.tate, info.rect.yoko);
    }
    return 0;
}
```